CANDY
DIALOGUE ENGINE

Player Choices Part 1

1.0.0

Player choices during dialogues are a common feature in video games. To achieve this, Candy DE features the §Choice_List command, which displays a list of choices for the player to select.

Overview

Categories

The §Choice_List command lets you group choices in categories, with a single category displayed at a time, and using specifically-configured choices to navigate.

The command also lets you designate whether choices are available at all (enabled/disabled), have any effects (active/inactive) or are visible (visible/hidden).

Choice Effects

When selected, choices perform a transition (e.g. Bridge, Jump, etc. -- see the **Transitions** guide for details) and run the spoken lines and commands in the block being transitioned to.

Choices can close the choice list entirely after being selected, or keep it open if the player should be allowed to make more choices.

As mentioned earlier, choices can also be configured to navigate to a different category.

Timers

Choice lists/menus can have multiple timers, with configurable countdown time and even number of loops. Timers can do two things when they reach 0: trigger a bridge transition, in order to run dialogue lines and commands, and/or select a choice.

UI

To display choices, you'll need 3 UI elements: a scene for the overall menu, scenes for categories, and scenes for choice buttons.

You can assign each §Choice_List command a general category scene and a general button scene, to use for all categories and choices, and then assign some categories or choices their own custom scenes.

The §Choice_List command, categories and choices can be assigned custom text strings that are passed to these UI elements. These strings are intended to act like arguments in a function, and allow you to design custom behavior on the UI side. For example, you may add code to the choice

button scenes to play a sound when clicked, and you can use each choice's custom string to indicate the sound to play.

Setup

At setup, before the choice list is even displayed, several dialogue blocks can be run, using bridge transitions (other transitions would defeat the purpose). Multiple such 'Setup' transitions can be ran: a general one for the entire command, one for each category, and one for each choice.

These 'Setup' transitions can be used to modify and configure the §Choice_List command, using dialogue commands. For example, you can use condition commands (e.g. §If) or the §Call command to modify which choices are available, or to randomize or limit the choices offered to the player.

Nesting

Nested choice lists are supported. Such nesting occurs if Setup transitions, or transitions triggered by selecting a choice trigger another §Choice_List command.

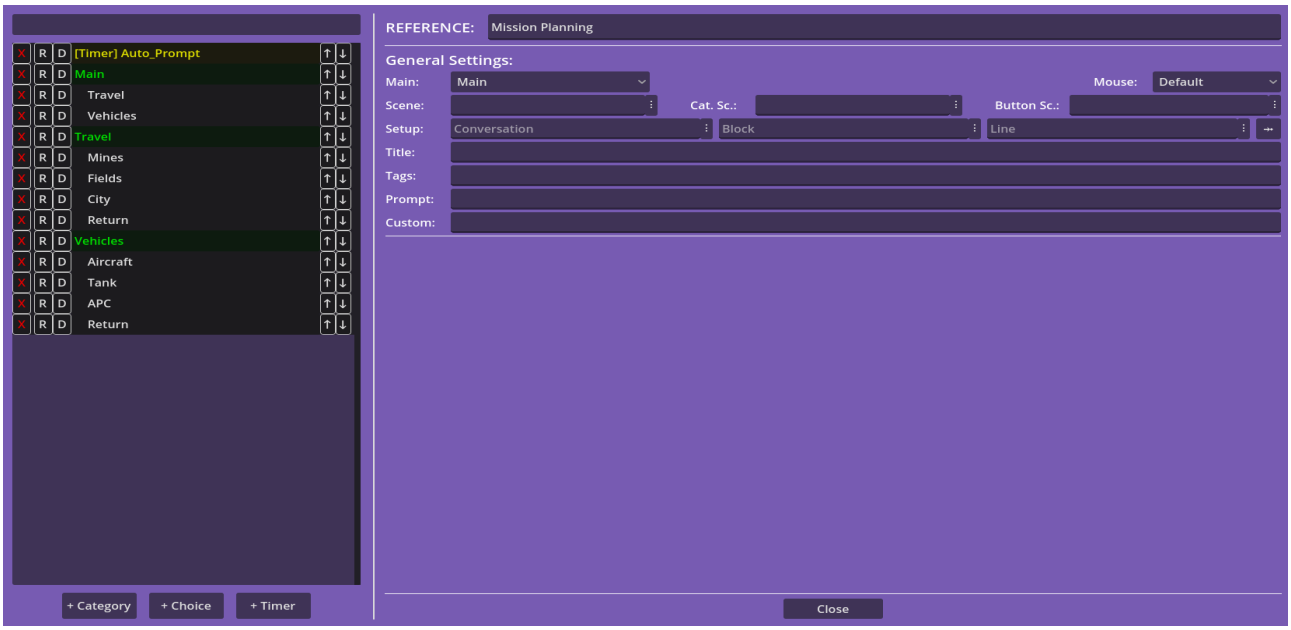
In such cases, the engine will store various data about each opened choice list, ensuring each list is cleanly separated from the others and allowing the engine to always return to previous lists once the newer ones close.

Overall, the §Choice_List command is simple to use, yet provides powerful options. Alternately, you can use the §Input command to display choice menus that work entirely on your own logic (see the Player Input guide).

§Choice_List Command

In the Candy Dialogue Creator, the §Choice_List command can be added by clicking the "Choice List" button under "Choices" in the command panel.

The command can then be configured by clicking the edit button, which opens the Choice List Editor window:



The area on the left lists all categories and choices, as well as timers, and allows you to add more or change their order.

To select a choice, category or timer, just click on it.

To move a choice, category or timer, click the arrow buttons next to it. To move by increments of 5, CTRL + Click the buttons. To move a choice to another category, SHIFT + Click the arrow buttons.

Timers can't be moved inside categories. Reordering them serves no technical purpose, except for your own readability/comfort.

The X button allows you to delete a choice or category, the R button renames a choice or category, and the D button duplicates a choice or category.

On the right, the top area labeled "REFERENCE" allows you to write a name, description, or keywords to identify the §Choice_List command. This has no effect in the dialogue engine: it only reminds you of the purpose of each choice list when your edit your dialogues.

The area below that displays settings for the choice list, split into four sections: General, Categories, Choices and Timers.

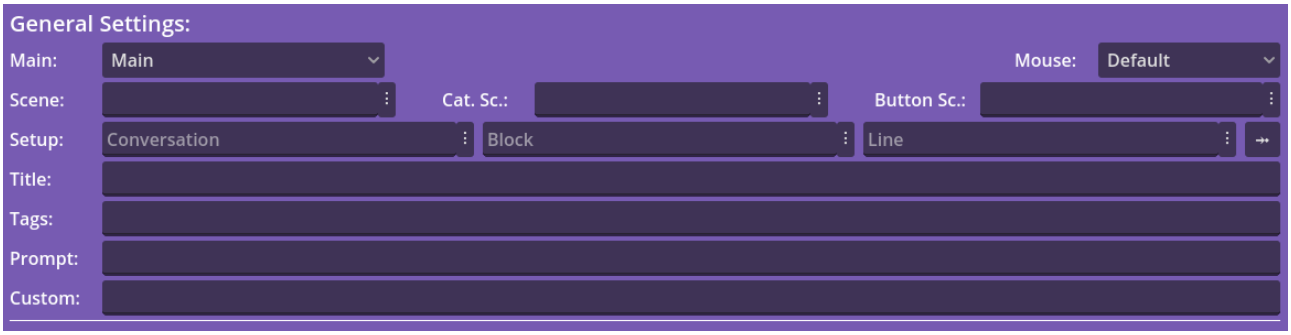
General Settings are top-level configuration options that relate to the choice list as a whole. Category Settings relate to individual choice categories, and Choice Settings relate to individual choices. Timer Settings are used to configure timers.

Before proceeding, it's important to understand how choice lists are displayed on screen to the player, by default:

When a §Choice_List command is processed, a "Menu" scene is instantiated for the entire list. Then, a "Category" scene is instantiated for each category, as a child of the menu scene. Finally, a "Button" scene is instantiated for each choice, and typically consists of a clickable button. These button scenes are children of their respective category scene.

Only one category is ever displayed on screen at a time, and choices can be configured to switch to any specific category.

General Settings



The General Settings section contains configuration options for the entire choice list.

Title -- A title given to the specific choice list, for future referencing.

Mouse -- The Mouse_Mode to change to when the list is first displayed.

Tags -- Optional tags that can help other commands target the specific choice list.

Scene -- The menu scene (.tscn file) to use for the choice list.

Cat. Sc. -- The default scene file to use for categories.

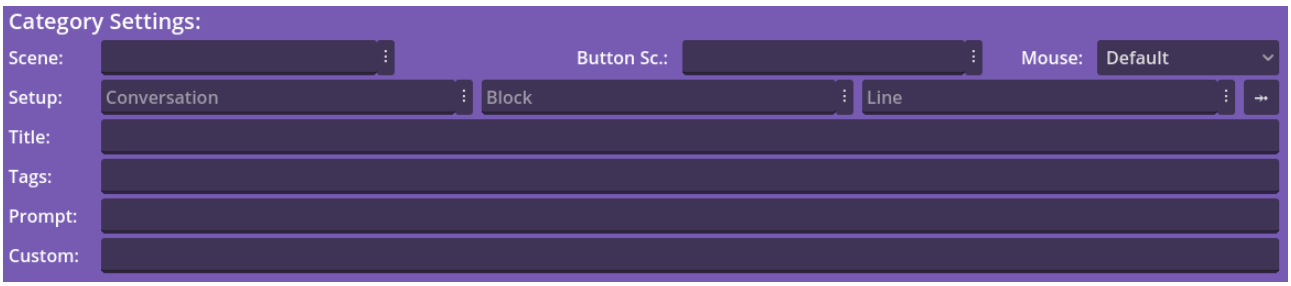
Button Sc. -- The default scene file to use for choice buttons.

Prompt -- An optional text to display to the player (e.g. "Choose an activity.").

Custom -- Optional custom data, for custom code you might add to choice lists.

Main -- The category that will be visible when the choice list is first displayed.

Category Settings



Mouse -- The Mouse_Mode to change to when the category is displayed.

Title -- A title for the category, which can be displayed to the player.

Tags -- Optional tags that can help other commands target the specific category.

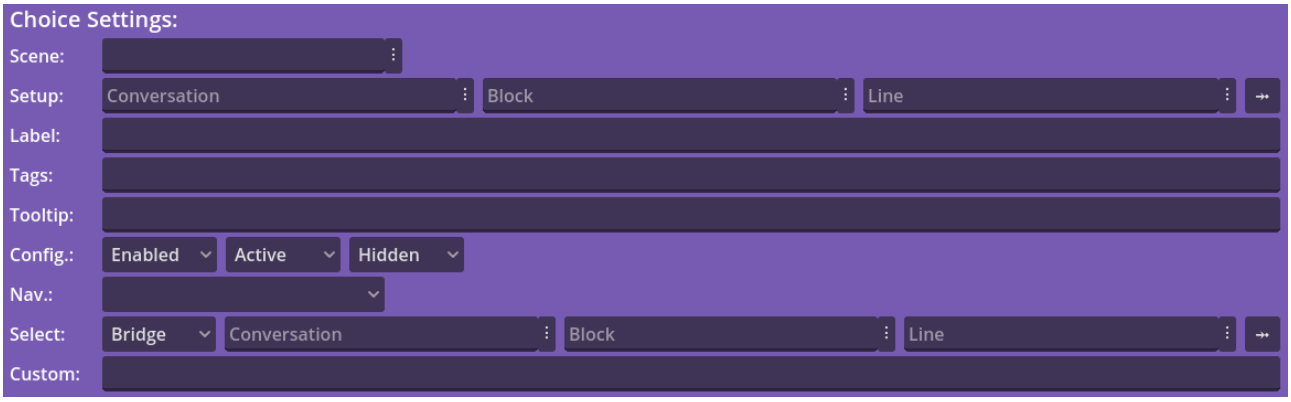
Scene -- The scene (.tscn file) to use for a given category. Overrides 'Cat. Sc.' in General Settings. This only needs to be used if you want some categories to use unique scene files.

Button Sc. -- The scene file that should be used for choice buttons in a given category. Overrides 'Button Sc.' in General Settings. This only needs to be used if you want some categories to use unique scene files for their choices.

Prompt -- An optional text to display to the player (e.g. "Choose an activity."). This is displayed in conjunction with the General Settings 'Prompt' option, it does not override it.

Custom -- Optional custom data, for custom code you might add to categories.

Choice Settings



Scene -- A specific button scene file (.tscn file) to use for a given choice. Overrides 'Button Sc.' in General Settings and in Category Settings. This only needs to be used if you want some choices to use unique scene files.

Label -- The text to display for the choice (e.g. "Yes", "No", "Tell me more.", etc.).

Tags -- Optional tags that can help other commands target the specific choice.

Tooltip -- Optional tooltip text to display for the choice.

Config. -- Whether the choice is enabled/disabled, active/inactive, visible/hidden.

Nav. -- Whether the choice changes the displayed category when selected. Set to '-' to disable.

Select -- Whether to run a transition to a Conversation/Block/Line, and whether to close the choice list. **"Continue"** runs a bridge transition and keeps the list open (no transition occurs if the Conversation, Block and Line fields are empty). **"Close"** closes the choice list without running a transition, making dialogue continue after the §Choice_List command. **"Bridge"**, **"Jump"**, **"Return"** and **"End"** perform the corresponding transition then close the choice list.

Custom -- Optional custom data, for custom code you might add to choice buttons.

Choice Configuration

Choices have three configurations: enabled/disabled, active/inactive and visible/hidden.

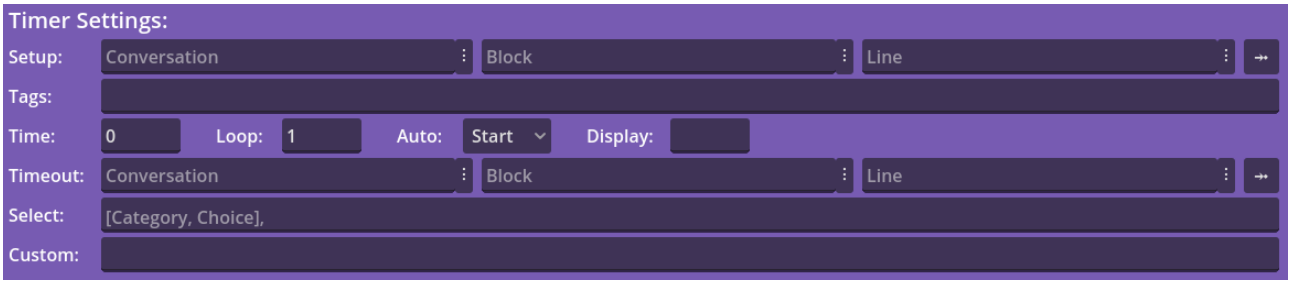
Disabled choices won't have a button created for them and won't be selected by timers.

Inactive choices will have a button, but clicking the button, or having it triggered by a timer, will have no effect.

Hidden choices will have a button, but its 'visible' property will be set to false, effectively hiding it. This can be used if you want timers to trigger choices that are not accessible to players, or for other gameplay reasons.

Note that the 'enabled' and 'active' status are unrelated to the 'active' property of Button nodes.

Timer Settings



Tags -- Optional tags that can help other commands target the specific timer.

Time -- The countdown time in seconds.

Loop -- The number of times the timer should countdown. A minimum value of 1 must be set, or -1 for infinite repeats.

Auto -- "Start" makes the timer automatically start when the choice menu is created on screen. "Hold" doesn't start it, in case you plan to start it later through other ways (e.g. a choice starting it, or a condition).

Display -- A value that indicates if/how the timer should be displayed on screen. You must write your own code in the Menu scene to display timers, so the Display value can mean anything you want it to.

Timeout -- When the countdown reaches 0, runs a Bridge transition to the indicated Block, for the purpose of running dialogue lines or commands. Leave all fields empty to disable.

Select -- The choice that the timer should trigger (as if selected by the player) when the countdown reaches 0. This is performed after the Timeout bridge transition above. A single can be triggered, and must be written in the following format: "Category, Choice" (separated by a comma, without quotation marks). Leave empty to disable.

Custom -- Optional custom data, for custom code you might add to choice buttons.

Note that when a choice is selected or a timer's countdown reaches 0, all timers are paused while the results are processed. If a timer triggers choices, the timer resumes after all choices have been processed. Timers which were paused through other means won't be restarted, so there are no risks of conflicts.

When multiple choices are triggered by a timer, they are queued and triggered in the order they appear in the Custom field. Choices can modify those after them, if configured to do so (such as by using commands like §Choice_Status, or other means). This is important to keep in mind to avoid unintended behaviors and conflicts.

Choices triggered by a timer can also modify the timer itself (such as by using the §Timer_Status command).

Setup Transitions

All Settings sections listed above also feature a 'Setup' option. This is an optional Bridge transition that occurs before the choice list is displayed (i.e. before the menu/category/choice button scenes are even instantiated).

This is meant to be used if some choices should be configured differently based on various conditions. In the transition Block, add condition commands (e.g. §If) and use §Choice_Status, §Set, §Call or other such commands to modify choices.

You can include other transitions in the Block, but be mindful that §Jump is a permanent transition and will therefore close the choice list entirely. Unless this is the behavior you want, you should use §Bridge and §Return transition commands only.

Keep in mind the order the 'Setup' transitions are performed in: The General 'Setup' transition is performed first, followed by the Category 'Setup' transitions. Choice 'Setup' transitions are performed last. Individual Category and Choice 'Setup' transitions are performed in the order they appear in the list.

If you don't want to run a Setup transition, simply leave the fields empty. See the Transitions guide for more information.

UI Elements

As mentioned previously, the §Choice_List command requires three types of scenes in your project's resource folders: a menu scene, a category scene, and a button scene. By default, these scenes should respectively be added in **Candy_DE/Scenes/Choices/Menus**, **Candy_DE/Scenes/Choices/Categories** and **Candy_DE/Scenes/Choices/Buttons**.

The **menu scene** is essentially the root of a choice list: it holds the category scenes as children. It features an important exported variable: **never_hide**. By default, when nested choice menus occur, the older menus are hidden automatically (and made visible again once the dialogue returns to them). If **never_hide** is set to 'true', the menu won't be hidden.

The **category scenes** hold button scenes as children, where each button represents a choice in that category.

Button scenes represent the available choices, and are usually buttons (or possibly other kinds of interactive nodes: you could theoretically trigger choices by pressing keys, typing text, detecting mouse_entered, or physics collisions).

To learn how to customize those UI scenes, see the **Choice UI Customization** guides.

An important thing to consider is that the dialogue engine doesn't care what happens inside your menu, category or choice button scenes. Once it has instantiated the choice menu, it awaits the choice_list_event signal, which is emitted by the **_choice_selected()** function in button scripts. When that signal is received, the engine processes whatever the choice is supposed to do.

This means that you can add plenty of custom code to choice UI scenes to modify how they look or behave.

The default code is enough for a standard choice menu like you find in most games, but the mostly 'barebones' structure makes it easy for developers to build upon and implement far more advanced features: since it's so minimalist, there isn't much that you need to understand or account for before customizing it.

Engine Settings

On a final note, **Candy_Engine.gd** features a **hide_choice_lists** variable, which controls whether older choice menus are hidden by newer nested menus. If 'true', menus are automatically hidden, unless their **never_hide** variable is 'true'. If **hide_choice_lists** is 'false', menus are never hidden.